

元学习（二）：万物皆可 Meta

从初始化、优化器、架构到数据处理与 few-shot 应用



基于李宏毅老师《机器学习 2025》课程内容整理

2026 年 6 月 9 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili)

时长：约 31 分 36 秒

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=56>

目录

1 这一讲的问题：学习算法里什么可以学	3
1.1 本章小结	4
2 Learning to Initialize: MAML 与 Reptile	4
2.1 本章小结	6
3 MAML 与 Pre-training: 名字不同，边界要看清	6
3.1 本章小结	8
4 Learning to Optimize: 优化器也可以学	8
4.1 本章小结	10
5 Network Architecture Search: 结构也可以 Meta	10
5.1 本章小结	13
6 Data Processing: 资料处理也可以学	13
6.1 本章小结	15
7 Beyond Gradient Descent: 学习与分类可以合在一起	15
7.1 本章小结	16
8 应用: Few-shot Image Classification 与 Omniglot	16
8.1 本章小结	20
9 总结与延伸	20
9.1 本章小结	21

1 这一讲的问题：学习算法里什么可以学

P55 建立了元学习的基本框架：普通机器学习学习函数 f_θ ，元学习学习学习算法 F_ϕ 。P56 进一步追问：一个学习算法里面到底哪些部分可以被放进 ϕ ？课程标题“万物皆可 Meta”不是说什么都随便套元学习，而是说许多原本靠人工设定的训练环节，都可以被写成可学习对象。



图 1: P56 的核心问题：What is learnable in a learning algorithm? 也就是在一个学习算法中，哪些组件可以由跨任务经验学习出来。¹

先把普通梯度下降写成一个流程。给定网络结构、初始参数、训练资料和更新规则，算法反复计算梯度并更新参数：

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} \ell(\theta^t; \mathcal{D}^{\text{train}}).$$

在这个式子里，很多东西都不是数据自动决定的：网络结构要人选，初始参数通常随机抽样，学习率 η 要调，优化器形式要选，训练资料还要经过 augmentation 或 weighting。

¹视频画面时间区间：00:00:00–00:00:15。



2

因此，本讲可以看成一张“可 Meta 化对象”的地图：

- 学初始参数: learning to initialize, 代表方法是 MAML。
- 学优化器: learning to optimize, 学习更新规则、学习率、动量等。
- 学网络架构: network architecture search, 把结构选择也视为可优化对象。
- 学数据处理: data augmentation、sample reweighting、课程式采样策略等。
- 学分类过程: metric-based meta-learning, 把训练和测试合成一个可学习函数。

“Meta” 的统一视角

判断一个方法是否属于元学习，不要只看它是否叫 MAML 或 few-shot。更稳的判断是：它是否利用许多 tasks 来学习某种“如何学习”的规则，并希望这条规则迁移到新 task 上。

1.1 本章小结

P56 的主线是把学习算法拆开：初始化、优化器、架构、数据处理、样本权重、分类流程都可以成为 ϕ 。元学习的实质是把这些原本手工设定的组件，改成可由 task-level 经验训练出来的对象。

2 Learning to Initialize: MAML 与 Reptile

第一类方法学习初始参数。普通训练中, 初始参数通常从固定随机分布抽样; learning to initialize 认为初始点本身可以被学习。如果初始点选得好, 新任务只需少量梯度更新就能得到不错模型。

²视频画面时间区间: 00:01:15-00:01:30。

Learning to initialize

- Model-Agnostic Meta-Learning (MAML)



Mammals

Chelsea Finn, Pieter Abbeel, and Sergey Levine, "Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks", ICML, 2017

- Reptile



<https://arxiv.org/abs/1803.02999>

图 3: Learning to initialize 的代表方法包括 MAML 与 Reptile: 它们学习一个适合快速适应的初始参数, 使模型面对新任务时可以用少量样本和少数更新步骤完成适应。³

MAML 的全名是 Model-Agnostic Meta-Learning。Model-agnostic 表示它不强依赖某种特定网络结构, 只要模型可以用梯度下降更新, 就可以尝试套用。一个简化的一步 MAML 过程如下。

$$\theta'_n = \phi - \alpha \nabla_{\phi} \ell_{\text{train}}^n(\phi).$$

其中 ϕ 是共享初始化, θ'_n 是第 n 个 task 经过一次 inner update 后的参数。然后用该 task 的 query set 评价:

$$L(\phi) = \sum_{n=1}^N \ell_{\text{query}}^n(\theta'_n).$$

最后对 ϕ 做 outer update。训练时, 优化器不是只看某个 task 的 training loss, 而是寻找一个初始化, 使它经过少量 task-specific 更新后能在 query examples 上表现好。

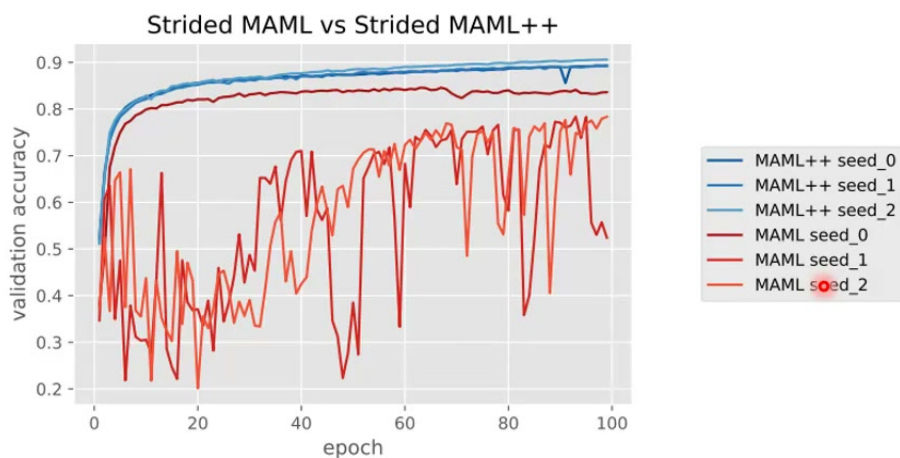
MAML 的直觉

MAML 不直接要求 ϕ 本身就是所有 task 的最佳参数, 而是要求 ϕ 离许多 task 的好解都不远。它学习的是“容易被少量资料推到正确方向”的起点。

训练 MAML 本身也要做优化, 而且可能并不稳定。课程引用了 How to train your MAML 一类工作, 说明训练这种初始化并不是简单跑一次梯度下降就好; inner loop 步数、学习率、二阶梯度近似、任务采样、归一化等都会影响稳定性。

³视频画面时间区间: 00:02:15–00:02:30。

How to train your Dragon MAML



Antreas Antoniou, Harrison Edwards, Amos Storkey, How to train your MAML, ICLR, 2019

图 4: MAML 训练可能非常敏感: 不同随机种子或设置会带来不同曲线, 说明“学习初始化”本身也是一个需要精心优化的 meta-training 问题。⁴

MAML 不是魔法初始化

MAML 学到的是跨 training tasks 的好起点。如果 training tasks 与未来 testing task 分布差异很大, 或者 inner update 的训练资料太少、噪声太大, 快速适应不一定可靠。它的成功依赖 task distribution 的相似性。

2.1 本章小结

Learning to initialize 把初始参数放进 ϕ 。MAML 的目标不是直接找一个万能分类器, 而是找一个能被少量新任务样本快速调整的起点。它把普通梯度下降嵌入 meta-training, 因此表达力强, 但训练也更敏感。

3 MAML 与 Pre-training: 名字不同, 边界要分清

课程专门比较 MAML 与 pre-training。二者都可能产生一个“好的初始化”, 但训练信号不同。Pre-training 往往在大量样本或 proxy task 上先训练一个模型, 再把它迁移到下游任务; MAML 则显式模拟“看 support set 后快速适应到 query set”的过程。

⁴视频画面时间区间: 00:03:30-00:03:45。

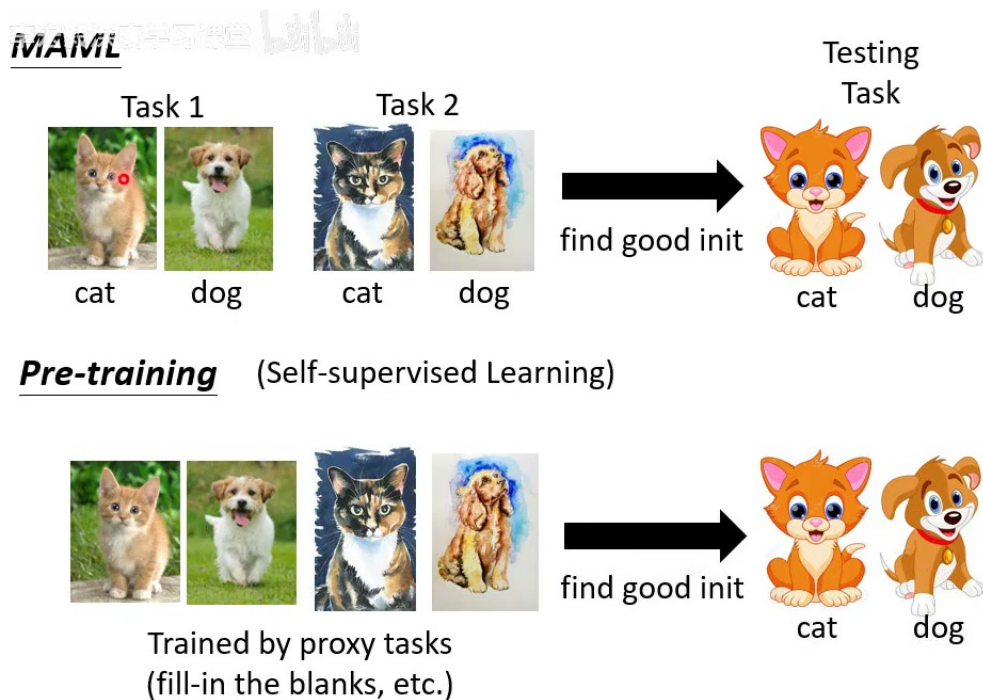


图 5: MAML 与 pre-training 都可能给测试任务提供一个好起点；差异在于 MAML 用 task/episode 结构训练快速适应，而典型 pre-training 则先在 proxy tasks 或大规模资料上训练模型。⁵

这一区别并不总是泾渭分明。现代 self-supervised pre-training、multi-task learning、domain adaptation 和 meta-learning 之间有很多交叠。课程提醒：不要过度拘泥于术语标签，更应看方法背后的训练目标与使用方式。

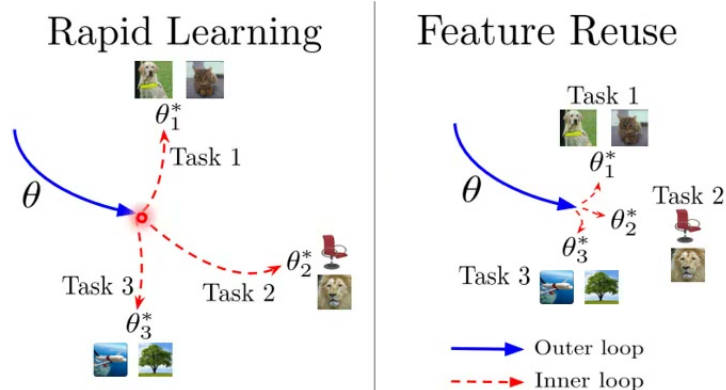
如果一种方法训练时没有显式模拟新 task 的少样本适应，而只是用很多资料学一个通用表示，它更像 pre-training 或 multi-task learning；如果它训练时不断采样 task，把 support/query 的适应和评价作为目标，就更像 meta-learning。

MAML 为什么有效也有两种解释：一种认为它真的学到了 rapid learning 的好初始化；另一种认为它主要学到了可以复用的 representation，inner loop 只是在已有特征上做少量调整。ANIL 相关研究指出，即使几乎不更新内层特征，效果也可能很好，这支持 feature reuse 的解释。

⁵视频画面时间区间：00:06:00–00:06:15。

MAML is good because

- ANIL (Almost No Inner Loop)



Aniruddh Raghu, Maithra Raghu, Samy Bengio, Oriol Vinyals, Rapid Learning or Feature Reuse? Towards Understanding the Effectiveness of MAML, ICLR, 2020

图 6: MAML 的效果可能来自两个机制: rapid learning 认为初始化便于快速移动到新 task 好解, feature reuse 则认为主要价值来自可迁移表示, inner loop 可能只需调整少量头部参数。⁶

看机制，不只看名字

当一个方法声称是 meta-learning 时，应追问它到底学习了什么：是学习初始化、表示、距离函数、优化器，还是只是在多任务资料上预训练？这些机制可能组合在一起，单靠方法名很难判断。

3.1 本章小结

MAML 与 pre-training 都可能产出可迁移起点，但 MAML 的目标更强调“经过少量 task 内更新后在 query set 上表现好”。实际研究中两者边界会模糊，理解时应看训练结构、适应过程和泛化目标，而不是只看论文标签。

4 Learning to Optimize：优化器也可以学

第二类可学习对象是 optimizer。普通梯度下降的更新规则中有许多人为设定：学习率、动量、衰减、Adam 的动量系数、RMSProp 的平滑系数等。Learning to optimize 问的是：能不能让机器从许多优化任务中学习一个优化器？

⁶ 视频画面时间区间：00:11:00-00:11:15。

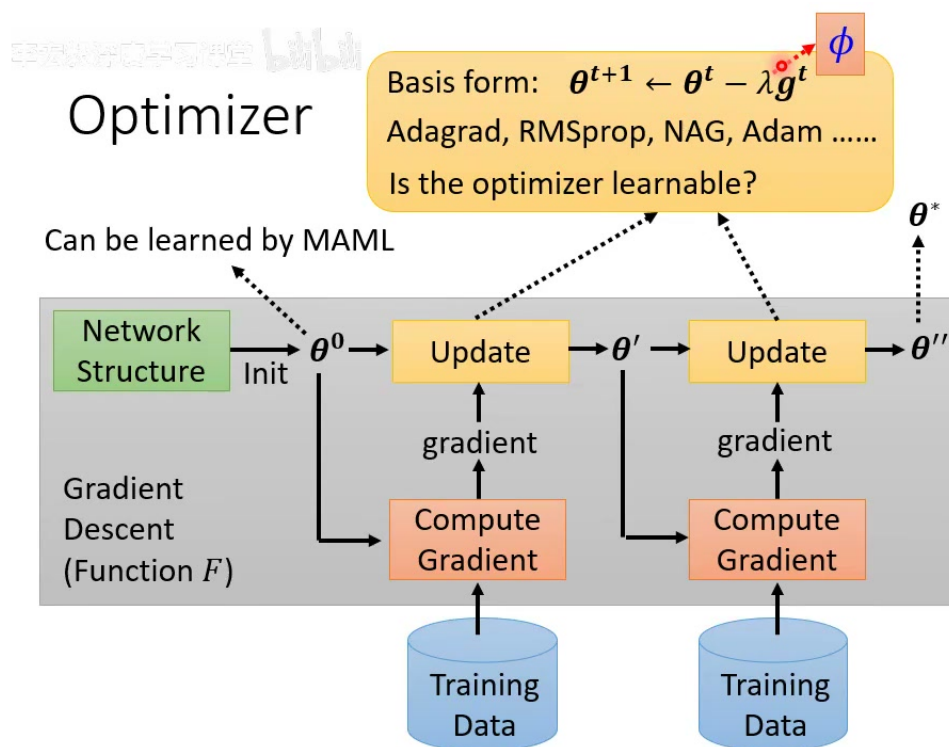


图 7: Optimizer 也可以被元学习：学习率、动量、更新公式或优化器内部状态都可以成为 ϕ ；学习目标是让这个 optimizer 在一批任务上带来更快或更稳定的下降。⁷

一个抽象写法是：

$$\theta^{t+1} = \theta^t + g_{\phi}(\nabla_{\theta} \ell(\theta^t), h^t),$$

其中 g_{ϕ} 是可学习更新器， h^t 是优化器状态，例如过去梯度、动量或隐藏状态。传统优化器的更新规则由人写死；learned optimizer 则从经验中学习“看到梯度后该怎么动”。

课程提到早期工作 Learning to learn by gradient descent by gradient descent。它们会在一些训练优化任务上学优化器，再测试能否迁移到不同网络、不同层数或不同数据集。

⁷ 视频画面时间区间：00:13:15–00:13:30。

Optimizer

Marcin Andrychowicz, et al., Learning to learn by gradient descent by gradient descent, NIPS, 2016

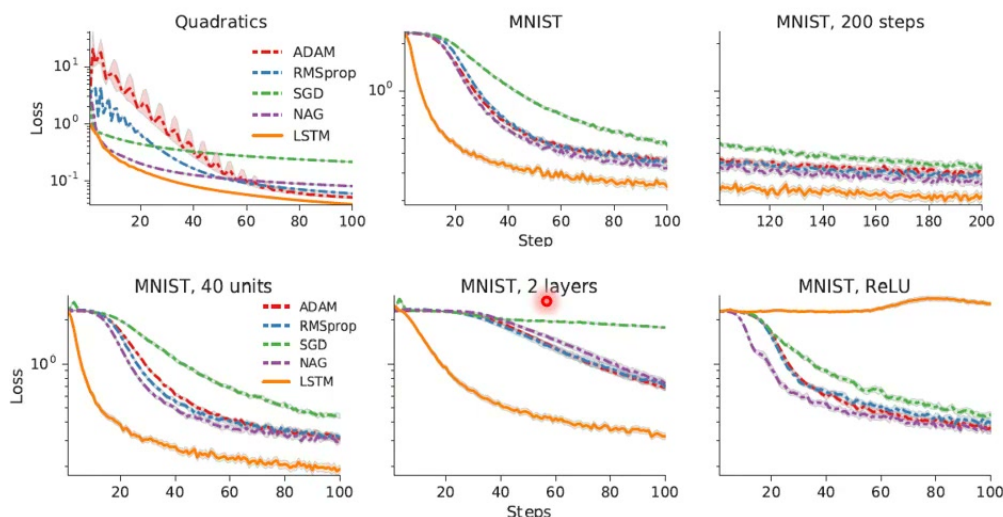


图 8: Learned optimizer 的实验曲线: 优化器可在若干训练任务上学习更新策略, 并尝试迁移到不同网络或数据集; 关键问题是泛化能力, 而不只是训练任务上的下降速度。⁸

学优化器的主要风险

学到的 optimizer 可能过拟合训练任务, 只在见过的 loss landscape 上好用; 也可能在长时间训练时不稳定。评价 learned optimizer 时, 必须看它是否能迁移到新网络、新资料、新 loss 形状, 而不是只看训练任务曲线。

4.1 本章小结

Learning to optimize 把更新规则本身放进 ϕ 。它不再只问“参数应该怎么变”, 而是问“看到梯度和状态以后, 优化器应该怎样产生更新”。这一路线很强, 但泛化与稳定性是核心难点。

5 Network Architecture Search: 结构也可以 Meta

第三类对象是网络结构。传统上, 网络架构由人设计: 层数、宽度、卷积核、skip connection、attention block 等都需要选择。Network Architecture Search (NAS) 把架构选择变成优化问题: 找一个结构, 使训练出来的网络在验证或测试任务上表现好。

在强化学习式 NAS 中, 一个 agent 逐步产生架构决策, 例如选择 filter size、stride、layer width、skip connection 等。这个 agent 的参数就是 ϕ , 它形成一个网络后, 网络再经过 within-task training, 最后用验证准确率或 loss 作为 reward 更新 agent。

⁸ 视频画面时间区间: 00:15:00-00:15:15。

Network Architecture Search (NAS)

$$\hat{\phi} = \arg \min_{\phi} L(\phi) \quad \nabla_{\phi} L(\phi) = ?$$

 Network Architecture

- Reinforcement Learning

- Barret Zoph, et al., Neural Architecture Search with Reinforcement Learning, ICLR 2017
- Barret Zoph, et al., Learning Transferable Architectures for Scalable Image Recognition, CVPR, 2018
- Hieu Pham, et al., Efficient Neural Architecture Search via Parameter Sharing, ICML, 2018

An agent uses a set of actions to determine the network architecture.

ϕ : the agent's parameters

$-L(\phi)$
Reward to be maximized

图 9: 强化学习式 NAS: agent 的参数 ϕ 决定网络架构, 生成架构后训练网络并取得表现, 把 $-L(\phi)$ 或准确率视为 reward 来更新 agent。⁹

把 NAS 放进元学习框架后, 可以看到两层训练:

- **Within-task training:** 给定某个架构, 训练该架构下的普通网络参数。
- **Across-task training:** 更新产生架构的 agent, 使它以后能提出更好的架构。

⁹视频画面时间区间: 00:17:30-00:17:45。

Network Architecture Search (NAS)

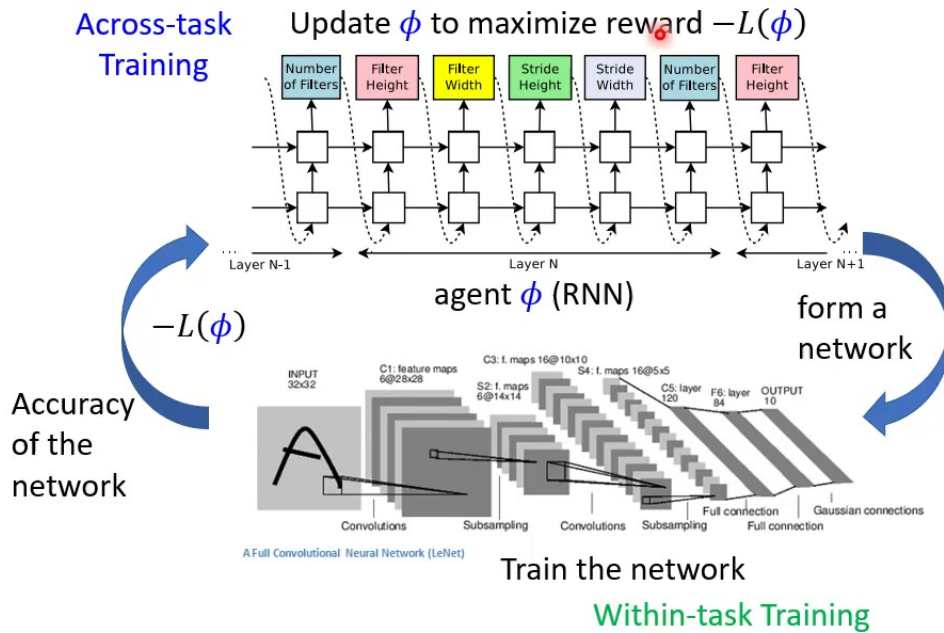


图 10: NAS 中的 across-task 与 within-task: agent 产生网络结构, 网络经过普通训练得到性能, 性能再反馈给 agent; 若训练与测试任务不同, NAS 更接近 meta-learning 设定。¹⁰

早期 NAS 常被质疑“训练和测试都在同一数据集上调结构”, 容易像是在同一 benchmark 上过度搜索。后来许多工作开始让 training tasks 与 testing tasks 不同, 评估搜索策略是否能迁移到新任务, 这就更符合元学习精神。

除了强化学习式 NAS, 还有可微分架构搜索 DARTS。DARTS 把离散结构选择放松成连续权重, 使 $\nabla_{\phi} L(\phi)$ 可以计算, 从而用梯度下降直接优化架构参数。

¹⁰ 视频画面时间区间: 00:19:45–00:20:00。

Network Architecture Search (NAS)

$$\hat{\phi} = \arg \min_{\phi} L(\phi) \quad \nabla_{\phi} L(\phi) = ?$$

ϕ $\searrow$ Network Architecture

- DARTS Hanxiao Liu, et al., DARTS: Differentiable Architecture Search, ICLR, 2019

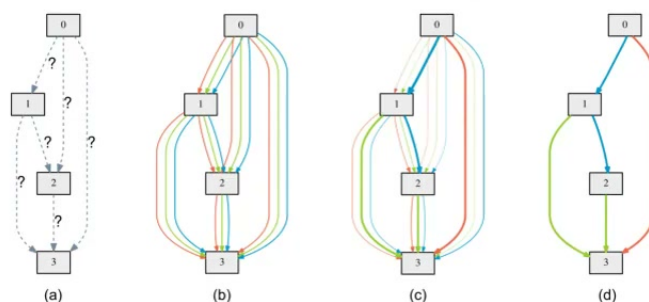


图 11: DARTS 将架构搜索近似成可微分问题, 让原本离散的 architecture choice 可以通过梯度下降优化; 这相当于把 network architecture 放进 ϕ 并令其可微。¹¹

5.1 本章小结

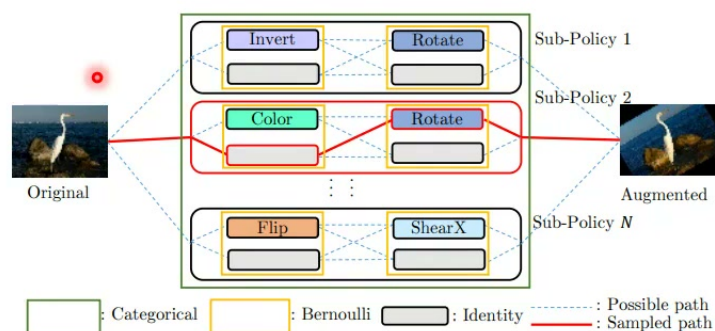
NAS 把网络结构从人工设计对象变成可学习对象。强化学习式 NAS 用 agent 产生架构并根据 reward 更新, DARTS 则把结构选择变得可微。它们说明“可学习组件”不必只限于参数或优化器, 连模型形状本身也可以被 meta-optimized。

6 Data Processing: 资料处理也可以学

除了模型和优化器, 训练资料的处理方式也可以 Meta。数据增强通常靠人工经验和试错: 是否翻转、裁切、旋转、改变颜色、加入噪声、混合样本, 都会影响模型泛化。既然 augmentation policy 会影响训练结果, 就可以把 policy 当成 ϕ 来学。

¹¹ 视频画面时间区间: 00:20:15–00:20:30。

Data Augmentation



Yonggang Li, Guosheng Hu, Yongtao Wang, Timothy Hospedales, Neil M. Robertson, Yongxin Yang, DADA: Differentiable Automatic Data Augmentation, ECCV, 2020

Daniel Ho, Eric Liang, Ion Stoica, Pieter Abbeel, Xi Chen, Population Based Augmentation: Efficient Learning of Augmentation Policy Schedules, ICML, 2019

Ekin D. Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, Quoc V. Le, AutoAugment: Learning Augmentation Policies from Data, CVPR, 2019

图 12: Data augmentation policy 也可被学习：系统从候选操作中组合增强策略，目标是让经过增强训练的模型在目标任务或验证集上表现更好。¹²

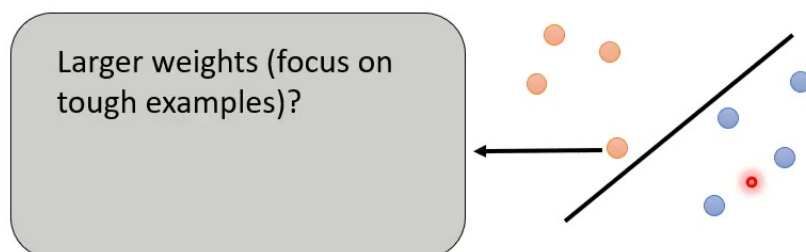
这类方法的典型结构是：先由 policy network 产生增强操作，再用增强后的数据训练模型，最后用模型表现更新 policy。它和 NAS 很像，只是搜索对象从 network architecture 变成 data transformation。

另一类资料处理是 sample reweighting。训练集中并非所有样本都应同等对待：边界附近样本可能更有信息量，也可能只是标签噪声；容易样本可以稳定训练，困难样本可以推动决策边界。到底给哪些样本更大权重，可以人工设定，也可以学习。

¹² 视频画面时间区间：00:20:45–00:21:00。

Sample Reweighting

- Give different samples different weights



Sample Weighting Strategies

Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, Deyu Meng,
Meta-Weight-Net: Learning an Explicit Mapping For Sample Weighting, NeurIPS, 2019
Mengye Ren, Wenyuan Zeng, Bin Yang, Raquel Urtasun, Learning to Reweight Examples
for Robust Deep Learning, ICML, 2018

图 13: Sample reweighting: 给不同样本不同权重。难样本可能值得更高权重，也可能是 noisy label 而应降低权重；权重策略本身可以由 meta-learning 学出。¹³

若把样本权重写成 $w_\phi(x, y)$ ，普通训练 loss 可改为

$$L(\theta; \phi) = \sum_{(x, y) \in \mathcal{D}^{\text{train}}} w_\phi(x, y) \ell(f_\theta(x), y).$$

外层目标则用验证集或 query set 来更新 ϕ 。这样，模型参数 θ 学分类器，meta-parameters ϕ 学“哪些样本更值得相信或强调”。

数据处理的 Meta 化很实用

许多工程问题的瓶颈不在网络结构，而在训练数据质量、增强策略、噪声标签和样本不平衡。把 augmentation 或 reweighting 学出来，往往比再堆一层网络更接近真实痛点。

6.1 本章小结

Data processing 也能成为 ϕ 。Data augmentation 学的是如何制造训练变化，sample reweighting 学的是哪些样本应被强调或削弱。它们把元学习从“模型内部”扩展到“训练资料流”的设计。

7 Beyond Gradient Descent: 学习与分类可以合在一起

到目前为止，很多方法仍保留“训练阶段”和“测试阶段”的分界：先用 training data 训练出某个参数 θ^* ，再把 θ^* 用在 testing data 上。课程接着问：能否把这个分界拿掉，让一个函数同时读入训练资料和测试资料，直接输出测试资料的分类结果？

¹³ 视频画面时间区间：00:21:15–00:21:30。

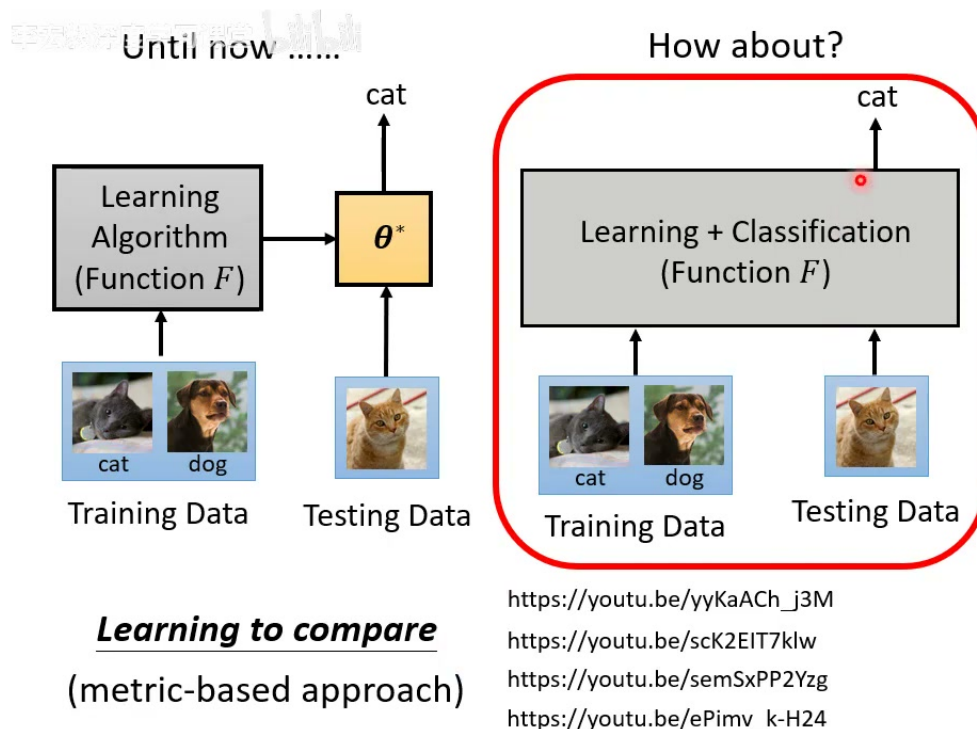


图 14: Learning to compare 或 metric-based meta-learning: 不一定先显式训练出 θ^* , 而是让一个函数同时接收 support examples 与 query example, 直接输出 query 的类别。¹⁴

这就是 metric-based meta-learning 的核心直觉。系统学习一个比较函数、嵌入空间或关系网络, 使 query example 可以与 support examples 比较。常见路线包括 Siamese Network、Matching Networks、Prototypical Networks、Relation Networks 等。

这类方法仍然符合元学习框架, 只是学习算法 F_ϕ 的形状不同。它不一定输出一个显式分类器参数 θ^* , 而是输出 query 的预测:

$$\hat{y}_q = F_\phi(\mathcal{D}^{\text{train}}, x_q).$$

其中 $\mathcal{D}^{\text{train}}$ 是 support set, x_q 是 query input。训练目标仍然来自许多 episodes: 给一组 support 和 query, 要求模型正确分类 query。

为什么 metric-based 方法适合 few-shot

当每类样本很少时, 完整训练一个高维分类器可能不稳定。Metric-based 方法改成学习“如何比较”, 让类别由 support examples 临时定义。新类别出现时, 不必重新训练整个网络, 只需把新类别 support examples 放进比较函数。

7.1 本章小结

Beyond gradient descent 的思路是: 学习算法不一定是“梯度下降产生参数”。Metric-based meta-learning 让模型直接学习 support-query 的比较机制, 把学习和分类合并进一个函数中。

8 应用: Few-shot Image Classification 与 Omniglot

元学习最经典的应用场景之一是 few-shot image classification。任务设定通常写成 N -way K -shot: 每个 episode 有 N 个类别, 每个类别只有 K 个 support examples。模型要根据这些少量样本

¹⁴ 视频画面时间区间: 00:24:30-00:24:45。

判断 query 属于哪个类别。



Few-shot Image Classification

- Each class only has a few images.



- **N-ways K-shot** classification: In each task, there are **N classes**, each has **K examples**.

图 15: Few-shot image classification 的 N -way K -shot 设定: 每个 task 有 N 个类别, 每类只有 K 个 support examples, 模型需判断 query 属于哪一类。¹⁵

经典 benchmark 是 Omniglot。它包含 1623 个字符类别, 每个字符有 20 个样本。由于类别多、每类样本少, 非常适合构造大量 few-shot tasks。

¹⁵ 视频画面时间区间: 00:26:00–00:26:15。



图 16: Omniglot 数据集包含 1623 个字符类别，每类约 20 个样本；在元学习里，它常被用作快速构造 few-shot classification episodes 的 benchmark。¹⁶

构造任务时，通常先把字符类别拆成 meta-training characters 与 meta-testing characters。然后从 meta-training characters 中随机抽取 N 个类别，每类抽取 K 个 support examples，再抽取 query examples，形成一个 training episode。Meta-testing 时则从完全未见过的 characters 中构造 episodes。

¹⁶ 视频画面时间区间：00:27:15–00:27:30。

	(A) Learning to initialize	(B) Learning to compare	(C) Other
Sound Event Detection	(Shi et al., 2020)	(Shimada et al., 2020a) (Chou et al., 2019) (Wang et al., 2020) (Shimada et al., 2020b) (Shi et al., 2020)	Network architecture search: (Li et al., 2020)
Keyword Spotting	(Chen et al., 2020a)	(Huh et al., 2020)	Net2Net: (Veniat et al., 2019) Network architecture search: (Mazzawi et al., 2019) Network architecture search: (Mo et al., 2020)
Text Classification	(Dou et al., 2019) (Bansal et al., 2019)	(Yu et al., 2018) (Tan et al., 2019) (Geng et al., 2019) (Sun et al., 2019)	Learning the learning algorithm: (Wu et al., 2019)
Voice Cloning			Learning the learning algorithm: (Chen et al., 2019b) (Serrà et al., 2019)
Sequence Labeling	(Wu et al., 2020)	(Hou et al., 2020)	
Machine Translation	(Gu et al., 2018) (Indurthi et al., 2020)		
Speech Recognition	(Hsu et al., 2020) (Klejch et al., 2019) (Winata et al., 2020a) (Winata et al., 2020b)		Learning to optimize: (Klejch et al., 2018) Network architecture search: (Chen et al., 2020b) (Baruwa et al., 2019)
Knowledge Graph	(Obamuyide and Vlachos, 2019) (Bose et al., 2019) (Lv et al., 2019) (Wang et al., 2019)	(Ye and Ling, 2019) (Chen et al., 2019a) (Xiong et al., 2018) (Gao et al., 2019)	
Dialogue / Chatbot	(Qian and Yu, 2019) (Madotto et al., 2019) (Mi et al., 2019)		Learning to optimize: (Chien and Licow, 2019)
Parsing	(Guo et al., 2019) (Huang et al., 2018)		
Word Embedding	(Hu et al., 2019)	(Sun et al., 2018)	
Multi-model		(Eloff et al., 2019)	Learning the learning algorithm: (Surís et al., 2019)

http://speech.ee.ntu.edu.tw/~tlkagk/meta_learning_table.pdf

图 18: Meta-learning 应用不局限于 Omniglot 或图像分类；课程列出语音、NLP 等领域中 learning to initialize、learning to compare 与其他 meta-learning 方法的应用。¹⁸

Benchmark 不能代表全部

Omniglot 很适合教学和快速验证，但真实任务往往有类别不平衡、域偏移、噪声标签、任务边界不清等问题。元学习方法能否落地，要看它是否能处理这些真实分布问题，而不只是 few-shot benchmark 分数。

8.1 本章小结

Few-shot image classification 给元学习提供了清晰实验场景：以 N -way K -shot episodes 训练学习算法，再测试其对新类别的快速适应。Omniglot 是典型 benchmark，但元学习思想也可迁移到语音、NLP 和其他任务中。

9 总结与延伸

P56 的核心不是介绍一个单独算法，而是把“学习算法里什么可以被学”系统展开。沿着梯度下降流程看，可学习对象从内到外不断扩大：

- 初始化可以学：MAML/Reptile 学一个适合快速适应的起点。
- 优化器可以学：学习率、动量、更新规则和优化器状态都可以由经验决定。
- 结构可以学：NAS 让 agent 或可微参数选择网络架构。
- 资料处理可以学：data augmentation 与 sample weighting 不必完全靠人工试错。
- 分类流程可以学：metric-based 方法把 support-query 比较本身变成可学习函数。

¹⁸ 视频画面时间区间：00:30:45–00:31:00。

这也解释了“万物皆可 Meta”的准确含义：并不是所有问题都应该硬套元学习，而是当某个设计选择反复出现在许多任务中、且人工选择成本高时，就可以考虑把它提升为 task-level 学习对象。

判断一个组件是否值得 Meta 化

一个组件适合元学习，通常需要满足三个条件：它会显著影响任务表现；它在许多任务之间存在可复用规律；它的好坏可以通过 validation/query performance 反馈出来。若没有跨任务规律，meta-training 只会学到训练任务的偏好。

从 P55 到 P56，可以形成一个完整检查表。看到任何元学习方法时，先问：

1. 它学习的 ϕ 是什么？初始化、优化器、结构、资料处理、度量函数，还是其他？
2. 它的 training tasks 与 testing tasks 是否分开？
3. 它如何构造 support/query，如何计算 meta-loss？
4. 它声称的优势来自 rapid adaptation、feature reuse、better optimization，还是更好的 data policy？
5. 它是否真的泛化到新任务，还是只在同一 benchmark 上调得很好？

这份检查表能帮助我们避免两个误区。第一，不要把 few-shot benchmark 成功自动等同于通用 learning to learn；第二，不要把所有预训练、多任务、NAS 或数据增强都粗略叫作元学习。真正关键的是跨任务训练结构与可迁移学习规则。

9.1 本章小结

P56 展示了元学习的广度：学习对象可以是初始化、优化器、架构、数据处理、样本权重或比较函数。它把 P55 的抽象框架具体化为一组方法族，也提醒我们：元学习的价值在于学习可迁移的“如何学习”，而不是给任何技巧贴上 meta 的标签。